

Self-Hosted n8n + ngrok Setup Guide

A reference for standing up self-hosted n8n with a public ngrok tunnel — written from an actual setup session, including the mistakes that ate up the most time.

Why you need a tunnel at all

n8n's "Send and Wait" approval nodes (Gmail, Telegram, Slack, etc.) embed a resume link in the message they send. That link has to be reachable from wherever the recipient clicks it — usually a phone on a different network. If n8n only knows about `localhost:5678`, the link is dead on arrival. A tunnel gives you a real public HTTPS address that forwards to your local instance.

One-time setup

1. Get a free ngrok static domain

- Sign up at ngrok.com (free, no credit card needed for this part)
- Dashboard → Universal Gateway → Domains → claim your free static domain
- It'll look like `something-random.ngrok-free.dev` or `.ngrok-free.app` — **this is yours permanently**, it won't change between restarts
- Also grab your **authtoken** from Dashboard → Getting Started → Your Authtoken

⚠ **Don't confuse the authtoken with a credential ID.** ngrok's dashboard shows resource IDs prefixed like `cr_...` (credentials) or `rd_...` (reserved domains) next to various settings. The actual authtoken is a separate, longer secret string — not one of these short prefixed IDs. Using the wrong one fails with `ERR_NGROK_105: authentication failed`.

2. Project files

You need two files in your project folder: `docker-compose.yml` and `.env`.

docker-compose.yml:

```
services:
  n8n:
    image: docker.n8n.io/n8nio/n8n
    container_name: n8n
    restart: unless-stopped
    ports:
      - "5678:5678"
    environment:
      - N8N_HOST=${N8N_HOST}
      - N8N_PROTOCOL=https
      - N8N_WEBHOOK_URL=https://${N8N_HOST}
      - N8N_PORT=5678
      - N8N_LICENSE_ACTIVATION_KEY=${N8N_LICENSE_ACTIVATION_KEY}
    volumes:
      - n8n_data:/home/node/.n8n
```

```
ngrok:
  image: ngrok/ngrok:latest
  container_name: n8n-tunnel
  restart: unless-stopped
  depends_on:
    - n8n
  environment:
    - NGROK_AUTHTOKEN=${NGROK_AUTHTOKEN}
  command:
    - "http"
    - "--url=${NGROK_DOMAIN}"
    - "n8n:5678"

volumes:
  n8n_data:
    external: true # only if reusing an existing volume from a prior
  setup
```

Notes on this file:

- `N8N_WEBHOOK_URL` is the current variable name (as of n8n 2.x). `WEBHOOK_URL` still works but is deprecated and logs a warning on every startup.
- The ngrok service targets `n8n:5678`, **not** `localhost:5678` — containers on the same Compose network reach each other by service name, not localhost.
- `external: true` on the volume tells Compose to attach to an already-existing named volume instead of creating a new empty one. Only include this if you're migrating from a manual `docker run` setup and already have a volume with your data in it (check with `docker volume ls` first).

`.env`:

```
# Your ngrok static domain (no https:// prefix, no trailing slash)
N8N_HOST="your-domain.ngrok-free.dev"
NGROK_DOMAIN="your-domain.ngrok-free.dev"

# From ngrok dashboard > Getting Started > Your Authtoken
# NOT a cr... credential ID — see warning above
NGROK_AUTHTOKEN="your_real_authtoken_here"

# Only needed if you have an n8n Community Edition or paid license key
N8N_LICENSE_ACTIVATION_KEY="your_license_key_here"
```

Lock it down: `chmod 600 .env`

3. Start everything

```
docker compose up -d
```

This starts both containers together. Check status with `docker compose ps`, and tail logs for a specific service (not container name) with:

```
docker compose logs -f ngrok
docker compose logs -f n8n
```

Note: `docker compose logs` wants the **service name** from the YAML (`ngrok`, `n8n`), not the `container_name` you set (`n8n-tunnel`). Using the wrong one gives `no such service`.

Common problems and fixes

Symptom	Cause	Fix
Approval email links show <code>localhost:5678</code> on your phone	<code>WEBHOOK_URL/N8N_HOST</code> not set, or a known bug in n8n's "Send and Wait" node link generation	Set both env vars explicitly; if still broken with correct config, it's an upstream n8n bug — check for a patched version
<code>docker: invalid reference format</code> on <code>docker run</code>	Multi-line command with <code>\</code> line-continuations got flattened when pasted into the terminal	Paste as one single line, no backslashes
Domain shows the Apache2 default page instead of n8n	Something else routing to port 80 locally, a stale ngrok process still bound to the domain, or (rarer) a local <code>/etc/hosts</code> entry pointing the domain back at your own machine	Check <code>pgrep -a ngrok</code> for stray processes, <code>cat /etc/hosts grep ngrok</code> for hijacked DNS, and test from a completely different device/network to isolate whether it's local
<code>ERR_NGROK_105: authentication failed</code>	Used a credential ID (<code>cr_...</code>) instead of the real authtoken	Get the actual token from the ngrok dashboard's "Your Authtoken" page
<code>Conflict. The container name "/n8n" is already in use</code>	An old stopped container still holds that name (<code>docker ps</code> only shows <i>running</i> containers — use <code>docker ps -a</code> to see it)	<code>docker rm n8n</code> before recreating
Found orphan containers (<code>n8n_postgres</code>) warning	A container from a previous/different Compose setup (e.g. an earlier course/tutorial) that isn't part of your current compose file	Harmless if your real data lives elsewhere (verify with <code>docker exec n8n ls -la /home/node/.n8n</code> — a populated <code>database.sqlite</code> confirms SQLite is your actual data store, not Postgres). Clean up with <code>docker compose up --remove-orphans</code> once confirmed safe
ngrok URL shows a "Visit Site" warning	Expected behavior on ngrok's free tier for browser traffic (anti-phishing)	One-time click per browser (cookie suppresses it for 7 days). For

Symptom	Cause	Fix
interstitial in the browser	measure)	programmatic requests (<code>curl</code> , scripts), add header <code>ngrok-skip-browser-warning: true</code>
n8n logs show Failed to start Python task runner... Python 3 is missing	n8n's optional internal Python task runner isn't relevant unless you use Python code nodes	Safe to ignore unless you actually need Python nodes
n8n logs show [Rudder] error: EAI_AGAIN	n8n's own telemetry/analytics client failing a DNS lookup	Harmless, unrelated to your setup

Before recreating any container

Always check what you're about to lose first:

```
docker volume ls          # see what
volumes exist
docker inspect <container_name> --format '{{.Mounts}}'  # see what's
mounted where
```

If your data lives in a named volume (not the container's own writable layer), you can safely **stop + rm +** recreate the container as long as you re-attach the **same volume name**.

Security reminders

- Never paste real API keys, OAuth secrets, or authtokens into a chat, ticket, or anywhere outside a secrets manager or `.env` file with locked-down permissions. If you ever do, rotate them — treat them as compromised regardless of who saw them.
- Keep `.env` out of version control (`.gitignore` it) and run `chmod 600 .env`.
- If using n8n's Gmail/Google credentials, prefer entering them as Credentials in the n8n UI (encrypted in its database) over environment variables, unless you have a specific reason to externalize them.